

Pedro Torres

CS 499

09/19/2025

Milestone Two: Software Design/Engineering Enhancement

Artifact & Context (what/when)

DailyDrop is an Android app designed to help users track their weight and optionally send an SMS summary. The app was originally built for CS 360 and is now being enhanced as part of CS 499. This milestone focuses on software design and engineering improvements that strengthen code quality, safety, and long-term maintainability.

Why I chose this artifact / Skills showcased

I chose this artifact because it highlights several important software engineering skills. It allows me to demonstrate the use of View Binding, proper separation of concerns between the user interface and data handling, validation utilities, RecyclerView best practices, runtime permissions, and clean Android architectural patterns. These improvements tie directly to professional design practices such as creating safer APIs, enforcing encapsulation, and clarifying dependencies in the codebase.

What I enhanced & how it improves quality

The enhancements made to DailyDrop greatly improved its overall quality. I replaced findViewById calls with View Binding, which provides compile-time safety, reduces null pointer exceptions, and makes the code's intent clearer. I centralized validation logic in a dedicated InputValidator class, which now serves as a reusable single source of truth for non-empty checks, weight parsing, and phone number validation. I introduced a data layer with WeightRepository and the data.Weight model, which decouples the UI from SQL and ensures persistence logic is

testable. The RecyclerView implementation was improved with ListAdapter, DiffUtil, and stable IDs, allowing smoother updates and efficient rendering. To improve encapsulation, the adapter now uses getter methods instead of directly accessing fields, which makes future model changes easier to support. On the user experience side, I added empty state handling, consistent rendering of “lbs” in the presentation layer, a date picker, and polished toast messages. Finally, I implemented runtime SMS permission checks to ensure the app behaves safely within Android’s security model.

Outcomes covered

These enhancements clearly demonstrate strong software design and engineering outcomes. The project now shows modular design with separation across UI, data, and utility layers, along with clean interfaces and safer APIs. It also incorporates algorithms and data structures through practical use of list diffing with DiffUtil and strongly typed models. In terms of databases and security, the app employs structured access to SQLite with a limited surface area, while also respecting Android’s runtime permission requirements for SMS. If only Category One is emphasized, the software design and engineering improvements take center stage.

Reflection (what I learned / challenges)

Working on this milestone taught me valuable lessons about careful refactoring and code structure. I ran into package mismatches and class visibility issues, which highlighted the importance of maintaining consistent organization during refactoring. When I moved to using getters and setters, I discovered hidden assumptions in the adapter that needed to be corrected, reinforcing the value of encapsulation. I also learned the benefit of making gradual changes and keeping the code compiling successfully at each step, which helped prevent error cascades. Looking ahead, I would approach future projects by adopting a formal architecture such as

MVVM earlier in the process and incorporating unit tests sooner to catch issues before they spread.

Future enhancements (roadmap)

There are several enhancements planned for DailyDrop that will continue to build on this foundation. I intend to add unit tests for both the InputValidator and repository queries, including instrumented tests. I also want to expand the app with features like exporting and backing up data, editing entries, and providing simple analytics such as weekly weight change trends.

Moving toward MVVM with ViewModel and LiveData or Flow will improve state handling and scalability. Finally, switching weight storage to a numeric type with a dedicated unit column will allow for internationalization and better formatting flexibility.